Optimization-based Invariant Generation

Hao Wu

Institute of Software, CAS & UCAS

Program Analysis and Verification Workshop
PKU, May 25, 2025

Contents

Program Verification

- Consider a program C:

$$
\begin{array}{l}
x \leftarrow 2 \\
\text{while True do} \\
\quad x \leftarrow 2x - 1 \\
\text{end}
\end{array}
$$

- Question: How to reason about a program? For example, to prove that $x > 0$ always holds.
- **Hoare logic** by Hoare 1969.

Program Verification

- Consider a program C:

$$
\begin{array}{l}
x \leftarrow 2 \\
\text{while True do} \\
\quad x \leftarrow 2x - 1 \\
\text{end}
\end{array}
$$

- Question: How to reason about a program? For example, to prove that $x > 0$ always holds.
- **Hoare logic** by Hoare 1969.

Program Verification: Hoare Logic

- **Hoare Logic**: to insert logical assertions into programs:

$\{\}$(Pre-condition)
$x \leftarrow 2$
$\{x = 2\}$
while True do
  $x \leftarrow 2x - 1$
end
$\{x > 0\}$(Post-condition)

- Key Problem: How to reason about loops?
- The key of Hoare logic: **Loop (Inductive) Invariants**.

Program Verification: Hoare Logic

- **Hoare Logic**: to insert logical assertions into programs:

$\{\}$(Pre-condition)
$x \leftarrow 2$
$\{x = 2\}$
while True do
  $x \leftarrow 2x - 1$
end
$\{x > 0\}$(Post-condition)

- Key Problem: How to reason about loops?
- The key of Hoare logic: **Loop (Inductive) Invariants**.

Program Verification: Inductive Invariants

...

$\{x = 2\}$

while True do $\{?\}$

$x \leftarrow 2x - 1$

end

$\{x > 0\}$

- **Loop Invariant**: An assertion at the loop entrance that holds whenever the program reaches that point. (say, $x > 0$)
- **Loop Inductive Invariant**: A loop invariant that is inductive: If it holds in one iteration, then it still holds in the next iteration. (say, $x \geq 1$)

Program Verification: Verification Conditions

$\{Pre(x)\}$
while $B(x)$ do $\{Inv(x)\}$
  $x \leftarrow f(x)$
end
$\{Post(x)\}$

- $Pre(x) \implies Inv(x)$
- $Inv(x) \wedge B(x) \implies Inv(f(x))$
- $Inv(x) \wedge \neg B(x) \implies Post(x)$
- To automize program verification, one needs to find suitable invariants for each loop.

Program Verification: Verification Conditions

$\{Pre(x)\}$
while $B(x)$ do $\{Inv(x)\}$
  $x \leftarrow f(x)$
end
$\{Post(x)\}$

- $Pre(x) \implies Inv(x)$
- $Inv(x) \wedge B(x) \implies Inv(f(x))$
- $Inv(x) \wedge \neg B(x) \implies Post(x)$
- To automize program verification, one needs to find suitable invariants for each loop.

Existing Approaches for Program Invariant Generation

Roughly speaking, four categories:

- **Abstract Interpretation**: perform fixed-point iteration in abstract domains.
- **Recurrence Relation**: find closed-form solutions and derive their relations.
- **Constraint Solving** (also optimization-based or template-based): set a parametrized template and solve constraints for parameters.
- **Learning-based**
  - Counter-Example Guided Inductive Synthesis: Refine the invariant candidate using counter-examples.
  - Machine Learning: Represent the invariant by ML models and train on large datasets.

Existing Approaches for Program Invariant Generation

Roughly speaking, four categories:

- **Abstract Interpretation**: perform fixed-point iteration in abstract domains.
- **Recurrence Relation**: find closed-form solutions and derive their relations.
- **Constraint Solving** (also optimization-based or template-based): set a parametrized template and solve constraints for parameters.
- **Learning-based**
  - Counter-Example Guided Inductive Synthesis: Refine the invariant candidate using counter-examples.
  - Machine Learning: Represent the invariant by ML models and train on large datasets.

Existing Approaches for Program Invariant Generation

Roughly speaking, four categories:

- **Abstract Interpretation**: perform fixed-point iteration in abstract domains.
- **Recurrence Relation**: find closed-form solutions and derive their relations.
- **Constraint Solving** (also optimization-based or template-based): set a parametrized template and solve constraints for parameters.
- **Learning-based**
  - Counter-Example Guided Inductive Synthesis: Refine the invariant candidate using counter-examples.
  - Machine Learning: Represent the invariant by ML models and train on large datasets.

Existing Approaches for Program Invariant Generation

Roughly speaking, four categories:

- **Abstract Interpretation**: perform fixed-point iteration in abstract domains.
- **Recurrence Relation**: find closed-form solutions and derive their relations.
- **Constraint Solving** (also optimization-based or template-based): set a parametrized template and solve constraints for parameters.
- **Learning-based**
  - Counter-Example Guided Inductive Synthesis: Refine the invariant candidate using counter-examples.
  - Machine Learning: Represent the invariant by ML models and train on large datasets.

Existing Approaches for Program Invariant Generation

Roughly speaking, four categories:

- **Abstract Interpretation**: perform fixed-point iteration in abstract domains.
- **Recurrence Relation**: find closed-form solutions and derive their relations.
- **Constraint Solving** (also optimization-based or template-based): set a parametrized template and solve constraints for parameters.
- **Learning-based**
  - Counter-Example Guided Inductive Synthesis: Refine the invariant candidate using counter-examples.
  - Machine Learning: Represent the invariant by ML models and train on large datasets.

Existing Approaches for Program Invariant Generation

Roughly speaking, four categories:

- **Abstract Interpretation**: perform fixed-point iteration in abstract domains.
- **Recurrence Relation**: find closed-form solutions and derive their relations.
- **Constraint Solving** (also optimization-based or template-based): set a parametrized template and solve constraints for parameters.
- **Learning-based**
  - Counter-Example Guided Inductive Synthesis: Refine the invariant candidate using counter-examples.
  - Machine Learning: Represent the invariant by ML models and train on large datasets.

Extending to Continuous-Time Dynamical Systems

System dynamics given by Ordinary Differential Equations (ODEs):

$$\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x})$$

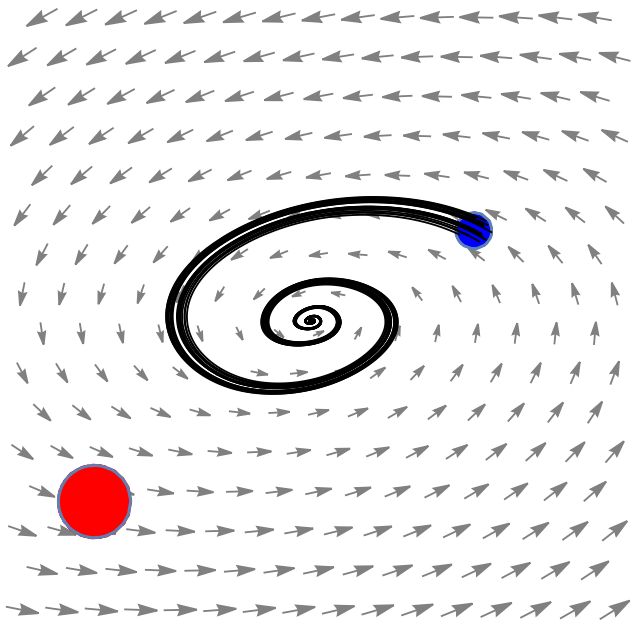
- $\boldsymbol{f} : \mathbb{R}^n \to \mathbb{R}^n$ is a locally Lipschitz continuous vector field.
- Unique trajectory $\xi_{\boldsymbol{x}_0} : \mathbb{R}_{\geq 0} \to \mathbb{R}^n$ for any initial state $\boldsymbol{x}_0$.

Safety Verification Problem of ODE Systems

Safety Verification Problem: Given domain $\mathcal{X}_d \subseteq \mathbb{R}^n$, initial set $\mathcal{X}_i \subset \mathcal{X}_d$, and unsafe set $\mathcal{X}_u \subset \mathcal{X}_d$ such that $\mathcal{X}_u \cap \mathcal{X}_i = \emptyset$. Prove that

$$\mathcal{X}_u \text{ is not reachable from any state in } \mathcal{X}_i.$$

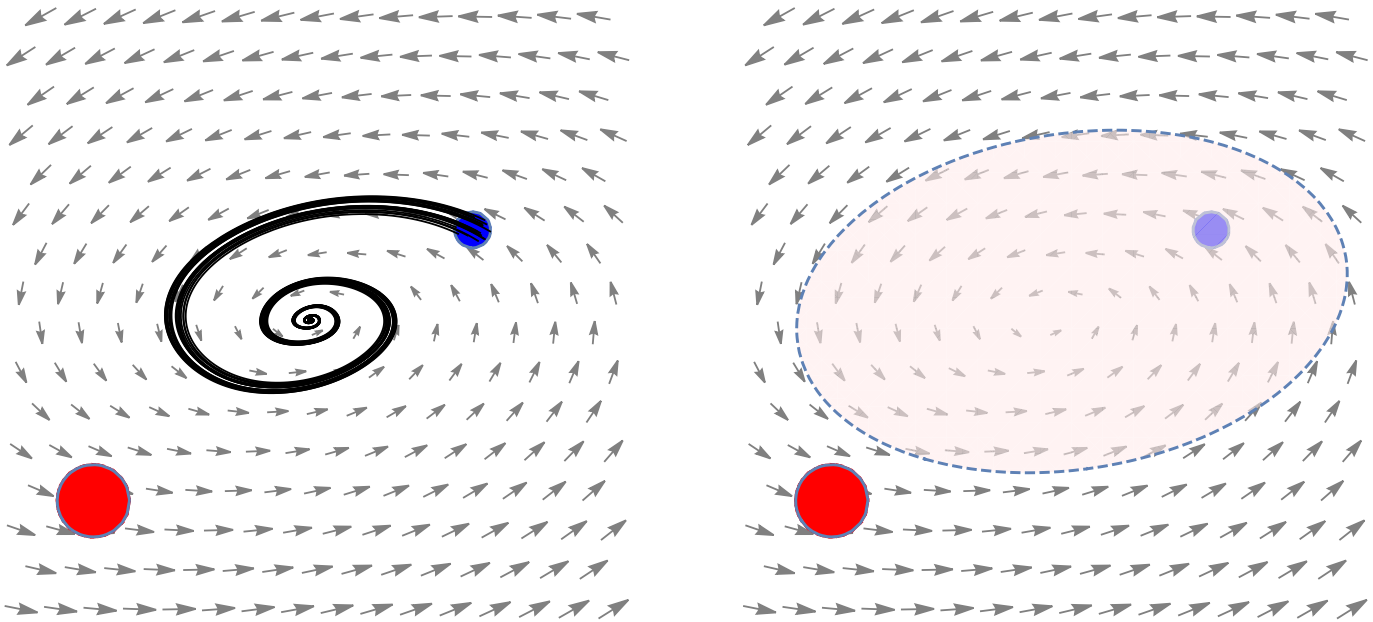
- $\mathcal{X}_i$: Pre-condition.
- $\mathcal{X}_u$: negation of Post-condition.
- $\mathcal{X}_d$: variable range



Differential Invariants

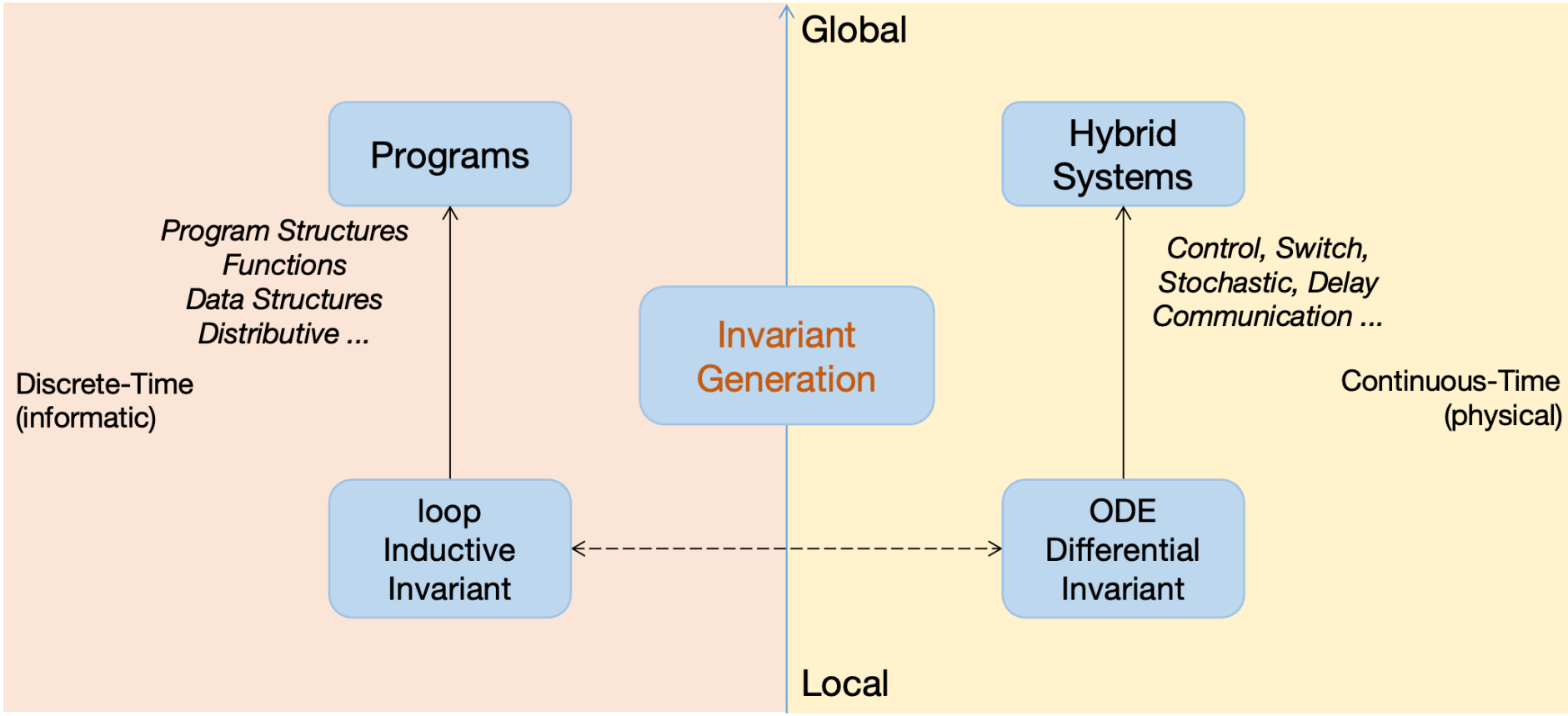
- Analogous to **inductive invariants** for programs.
- A **differential/positive invariant** is a subset $\Omega \subseteq \mathcal{X}_d$ such that any trajectory starting from Ω stays within Ω forever, i.e.,

$$\forall \boldsymbol{x}_0 \in \Omega, \forall t \in \mathbb{R}_{\geq 0}.\ \xi_{\boldsymbol{x}_0}(t) \in \Omega.$$



Invariant Generation

- The most challenging problem for **Hoare-logic**-style verification.



Optimization-based Invariant Generation

- Constraint Solving provides a unified framework for synthesizing invariants for both programs and continuous-time dynamical systems.
  1. Set a parameterized template for the target invariant.
  2. Encode the invariant conditions into constraints for parameters.
  3. Solve the constraints to obtain assignments to parameters.
- Assumptions:
  1. Program variables are $\mathbb{R}$-valued.
  2. All functions and predicates are polynomial (or semialgebraic).
  3. Pre-condition, post-condition, domain, initial set, and unsafe set are all basic semialgebraic sets (=defined by some polynomial (in)equalities).
  4. The invariant template is a polynomial with unknown coefficients and the target invariant is expressed as $I(\boldsymbol{x};\boldsymbol{a}) \leq 0$. For example

$$I(\boldsymbol{x};\boldsymbol{a}) = a_1x_1^2 + a_2x_1x_2 + a_3x_2^2 + a_4x_1 + a_5x_2 + a_6 \in \mathbb{R}[x_1, x_2],$$

Optimization-based Invariant Generation

- Constraint Solving provides a unified framework for synthesizing invariants for both programs and continuous-time dynamical systems.
  1. Set a parameterized template for the target invariant.
  2. Encode the invariant conditions into constraints for parameters.
  3. Solve the constraints to obtain assignments to parameters.
- Assumptions:
  1. Program variables are $\mathbb{R}$-valued.
  2. All functions and predicates are polynomial (or semialgebraic).
  3. Pre-condition, post-condition, domain, initial set, and unsafe set are all basic semialgebraic sets (=defined by some polynomial (in)equalities).
  4. The invariant template is a polynomial with unknown coefficients and the target invariant is expressed as $I(\boldsymbol{x};\boldsymbol{a}) \leq 0$. For example

$$I(\boldsymbol{x};\boldsymbol{a}) = a_1x_1^2 + a_2x_1x_2 + a_3x_2^2 + a_4x_1 + a_5x_2 + a_6 \in \mathbb{R}[x_1, x_2],$$

Conditions for Loop Inductive Invariants

- Sufficient and Necessary condition for **inductive invariants**:
  1. $\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ \mathrm{PreCond}(\boldsymbol{x}) \implies I(\boldsymbol{x}; \boldsymbol{a}) \leq 0,$
  2. $\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ I(\boldsymbol{x}; \boldsymbol{a}) \leq 0 \wedge g(\boldsymbol{x}) \leq 0 \implies I(f(\boldsymbol{x}); \boldsymbol{a}) \leq 0,$
  3. $\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ I(\boldsymbol{x}; \boldsymbol{a}) \leq 0 \wedge g(\boldsymbol{x}) > 0 \implies \mathrm{PostCond}(\boldsymbol{x}).$
- Decidable according to Tarski's theorem [Tarski, 1951] ($\boldsymbol{a}, \boldsymbol{x} \in \mathbb{R}$).
- SMT and Quantifier Elimination can not deal with such constraints efficiently.

Conditions for ODE Differential Invariants

- Sufficient condition for **differential invariants** (also termed barrier certificates):
  1. $\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ \boldsymbol{x} \in \mathcal{X}_i \implies B(\boldsymbol{x}; \boldsymbol{a}) \leq 0,$
  2. $\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ \boldsymbol{x} \in \mathcal{X}_d \wedge B(\boldsymbol{x}; \boldsymbol{a}) = 0 \implies \mathfrak{L}_{\boldsymbol{f}} B(\boldsymbol{x}; \boldsymbol{a}) < 0,$
  3. $\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ \boldsymbol{x} \in \mathcal{X}_u \implies B(\boldsymbol{x}; \boldsymbol{a}) > 0.$

  where $\mathfrak{L}_{\boldsymbol{f}} B(\boldsymbol{x}; \boldsymbol{a}) \ \hat{=}\ \langle \nabla B(\boldsymbol{x}; \boldsymbol{a}), \boldsymbol{f}(\boldsymbol{x}) \rangle$ is the Lie derivative.
- Sufficient and Necessary condition: need to consider higher order Lie derivatives.
  - First proposed in [Liu et al., 2011]
  - Utilized in optimization-based method [Wang et al., 2021].

Conditions for ODE Differential Invariants

- Sufficient condition for **differential invariants** (also termed barrier certificates):
  1. $\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ \boldsymbol{x} \in \mathcal{X}_i \implies B(\boldsymbol{x}; \boldsymbol{a}) \leq 0,$
  2. $\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ \boldsymbol{x} \in \mathcal{X}_d \wedge B(\boldsymbol{x}; \boldsymbol{a}) = 0 \implies \mathfrak{L}_{\boldsymbol{f}} B(\boldsymbol{x}; \boldsymbol{a}) < 0,$
  3. $\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ \boldsymbol{x} \in \mathcal{X}_u \implies B(\boldsymbol{x}; \boldsymbol{a}) > 0.$

  where $\mathfrak{L}_{\boldsymbol{f}} B(\boldsymbol{x}; \boldsymbol{a}) \ \hat{=}\ \langle \nabla B(\boldsymbol{x}; \boldsymbol{a}), \boldsymbol{f}(\boldsymbol{x}) \rangle$ is the Lie derivative.
- Sufficient and Necessary condition: need to consider higher order Lie derivatives.
  - First proposed in [Liu et al., 2011]
  - Utilized in optimization-based method [Wang et al., 2021].

Real Algebraic Geometry Tool: SOS polynomials

- A polynomial $\sigma(\boldsymbol{x})$ is **Sum-of-Squares** (SOS) if it can written as

$$\sigma(\boldsymbol{x}) = \sum_{i=1}^{n} p_i^2(\boldsymbol{x}) \in \mathbb{R}[\boldsymbol{x}].$$

- $\Sigma[\boldsymbol{x}]$ denotes the set of SOS polynomials.
- A polynomial $f(\boldsymbol{x})$ is SOS iff the Gram matrix of f is PSD.

$$\begin{aligned} f(x_1, x_2) &= 2x_1^4 + 2x_1^3x_2 - x_1^2x_2^2 + 5x_2^4 \\ &= \underbrace{\begin{pmatrix} x_1^2 \\ x_2^2 \\ x_1x_2 \end{pmatrix}'}_{\text{basis}} \underbrace{\begin{pmatrix} 2 & -3 & 1 \\ -3 & 5 & 0 \\ 1 & 0 & 5 \end{pmatrix}}_{\text{Gram matrix}} \underbrace{\begin{pmatrix} x_1^2 \\ x_2^2 \\ x_1x_2 \end{pmatrix}}_{\text{basis}} \\ &= \frac{1}{2}(2x_1^2 - 3x_2^3 + x_1x_2)^2 + \frac{1}{2}(x_2^2 + 3x_1x_2)^2 \end{aligned}$$

Real Algebraic Geometry Tool: SOS polynomials

- A polynomial $\sigma(\boldsymbol{x})$ is **Sum-of-Squares** (SOS) if it can written as

$$\sigma(\boldsymbol{x}) = \sum_{i=1}^{n} p_i^2(\boldsymbol{x}) \in \mathbb{R}[\boldsymbol{x}].$$

- $\Sigma[\boldsymbol{x}]$ denotes the set of SOS polynomials.
- A polynomial $f(\boldsymbol{x})$ is SOS iff the Gram matrix of f is PSD.

$$\begin{aligned} f(x_1, x_2) &= 2x_1^4 + 2x_1^3x_2 - x_1^2x_2^2 + 5x_2^4 \\ &= \underbrace{\begin{pmatrix} x_1^2 \\ x_2^2 \\ x_1x_2 \end{pmatrix}'}_{\text{basis}} \underbrace{\begin{pmatrix} 2 & -3 & 1 \\ -3 & 5 & 0 \\ 1 & 0 & 5 \end{pmatrix}}_{\text{Gram matrix}} \underbrace{\begin{pmatrix} x_1^2 \\ x_2^2 \\ x_1x_2 \end{pmatrix}}_{\text{basis}} \\ &= \frac{1}{2}(2x_1^2 - 3x_2^3 + x_1x_2)^2 + \frac{1}{2}(x_2^2 + 3x_1x_2)^2 \end{aligned}$$

Real Algebraic Geometry Tool: SOS polynomials

- A polynomial $\sigma(\boldsymbol{x})$ is **Sum-of-Squares** (SOS) if it can written as

$$\sigma(\boldsymbol{x}) = \sum_{i=1}^{n} p_i^2(\boldsymbol{x}) \in \mathbb{R}[\boldsymbol{x}].$$

- $\Sigma[\boldsymbol{x}]$ denotes the set of SOS polynomials.
- A polynomial $f(\boldsymbol{x})$ is SOS iff the Gram matrix of f is PSD.

$$\begin{aligned} f(x_1, x_2) &= 2x_1^4 + 2x_1^3x_2 - x_1^2x_2^2 + 5x_2^4 \\ &= \underbrace{\begin{pmatrix} x_1^2 \\ x_2^2 \\ x_1x_2 \end{pmatrix}'}_{\text{basis}} \underbrace{\begin{pmatrix} 2 & -3 & 1 \\ -3 & 5 & 0 \\ 1 & 0 & 5 \end{pmatrix}}_{\text{Gram matrix}} \underbrace{\begin{pmatrix} x_1^2 \\ x_2^2 \\ x_1x_2 \end{pmatrix}}_{\text{basis}} \\ &= \frac{1}{2}(2x_1^2 - 3x_2^3 + x_1x_2)^2 + \frac{1}{2}(x_2^2 + 3x_1x_2)^2 \end{aligned}$$

Real Algebraic Geometry Tool: Putinar's Positivstellensatz

- Let $S = \{x \mid p_i(x) \geq 0,\ i = 1, \ldots, m\}$.
- **Quadratic Module**

$$\mathbf{QM}(S) \triangleq \{\sigma_0 + \sum_{i=1}^{m} \sigma_i p_i \mid \sigma_i \in \Sigma[\boldsymbol{x}] \text{ for } 1 \leq i \leq m\}$$

- A quadratic module **QM** is **Archimedean** if $N - \|\boldsymbol{x}\|^2 \in \mathbf{QM}$ for some constant $N \in \mathbb{N}$.
  - $N - \|\boldsymbol{x}\|^2 \geq 0$ over S implies that S is bounded.

Theorem (Putinar's Positivstellensatz)

Assume the Archimedean condition holds. If $f \in \mathbb{R}[x]$ is strictly positive on S, then $f \in \mathbf{QM}(S)$, i.e.,

$$f(x) = \sigma_0(x) + \sum_{i=1}^{m} p_i(x) \cdot \sigma_i(x), \quad \sigma_i(x) \in \Sigma[x].$$

Real Algebraic Geometry Tool: Putinar's Positivstellensatz

- Let $S = \{x \mid p_i(x) \geq 0,\ i = 1, \ldots, m\}$.
- **Quadratic Module**

$$\mathbf{QM}(S) \triangleq \{\sigma_0 + \sum_{i=1}^{m} \sigma_i p_i \mid \sigma_i \in \Sigma[\boldsymbol{x}] \text{ for } 1 \leq i \leq m\}$$

- A quadratic module **QM** is **Archimedean** if $N - \|\boldsymbol{x}\|^2 \in \mathbf{QM}$ for some constant $N \in \mathbb{N}$.
  - $N - \|\boldsymbol{x}\|^2 \geq 0$ over S implies that S is bounded.

Theorem (Putinar's Positivstellensatz)

Assume the Archimedean condition holds. If $f \in \mathbb{R}[x]$ is strictly positive on S, then $f \in \mathbf{QM}(S)$, i.e.,

$$f(x) = \sigma_0(x) + \sum_{i=1}^{m} p_i(x) \cdot \sigma_i(x), \quad \sigma_i(x) \in \Sigma[x].$$

Real Algebraic Geometry Tool: Putinar's Positivstellensatz

- Let $S = \{x \mid p_i(x) \geq 0,\ i = 1, \ldots, m\}$.
- **Quadratic Module**

$$\mathbf{QM}(S) \triangleq \{\sigma_0 + \sum_{i=1}^{m} \sigma_i p_i \mid \sigma_i \in \Sigma[\boldsymbol{x}] \text{ for } 1 \leq i \leq m\}$$

- A quadratic module **QM** is **Archimedean** if $N - \|\boldsymbol{x}\|^2 \in \mathbf{QM}$ for some constant $N \in \mathbb{N}$.
  - $N - \|\boldsymbol{x}\|^2 \geq 0$ over S implies that S is bounded.

Theorem (Putinar's Positivstellensatz)

Assume the Archimedean condition holds. If $f \in \mathbb{R}[x]$ is strictly positive on S, then $f \in \mathbf{QM}(S)$, i.e.,

$$f(x) = \sigma_0(x) + \sum_{i=1}^{m} p_i(x) \cdot \sigma_i(x), \quad \sigma_i(x) \in \Sigma[x].$$

SOS Constraints for Inductive Invariants

- Take inductive invariants for example:
  1. $\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ p(\boldsymbol{x}) \leq 0 \implies I(\boldsymbol{x}; \boldsymbol{a}) \leq 0,$
  2. $\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ I(\boldsymbol{x}; \boldsymbol{a}) \leq 0 \wedge g(\boldsymbol{x}) \leq 0 \implies I(f(\boldsymbol{x}); \boldsymbol{a}) \leq 0,$
  3. $\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ I(\boldsymbol{x}; \boldsymbol{a}) \leq 0 \wedge g(\boldsymbol{x}) > 0 \implies q(\boldsymbol{x}) \leq 0.$

  where PreCond($\boldsymbol{x}$) is $p(\boldsymbol{x}) \leq 0$ and PostCond($\boldsymbol{x}$) is $q(\boldsymbol{x}) \leq 0$.
- By applying **Putinar's Positivstellensatz**, for arbitrarily small $\epsilon > 0$

$$
\begin{aligned}
&\text{find} \quad \boldsymbol{a}, \boldsymbol{b} \\
&-I(\boldsymbol{x}; \boldsymbol{a}) + \epsilon = \sigma_1(\boldsymbol{x}; \boldsymbol{b}) + \sigma_2(\boldsymbol{x}; \boldsymbol{b}) \cdot (-p(\boldsymbol{x})), \\
&-I(f(\boldsymbol{x}); \boldsymbol{a}) + \epsilon = \sigma_3(\boldsymbol{x}; \boldsymbol{b}) + \sigma_4(\boldsymbol{x}; \boldsymbol{b}) \cdot (-g(\boldsymbol{x})) + \sigma_5(\boldsymbol{x}; \boldsymbol{b}) \cdot (-I(\boldsymbol{x}; \boldsymbol{a})) \\
&-q(\boldsymbol{x}) + \epsilon = \sigma_6(\boldsymbol{x}; \boldsymbol{b}) + \sigma_7(\boldsymbol{x}; \boldsymbol{b}) \cdot g(\boldsymbol{x}) + \sigma_8(\boldsymbol{x}; \boldsymbol{b}) \cdot (-I(\boldsymbol{x}; \boldsymbol{a})) \\
&\sigma_1(\boldsymbol{x}; \boldsymbol{b}), \ldots, \sigma_8(\boldsymbol{x}; \boldsymbol{b}) \in \Sigma[\boldsymbol{x}]
\end{aligned}
$$

SOS Constraints for Inductive Invariants

- Take inductive invariants for example:
  1. $\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ p(\boldsymbol{x}) \leq 0 \implies I(\boldsymbol{x}; \boldsymbol{a}) \leq 0,$
  2. $\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ I(\boldsymbol{x}; \boldsymbol{a}) \leq 0 \wedge g(\boldsymbol{x}) \leq 0 \implies I(f(\boldsymbol{x}); \boldsymbol{a}) \leq 0,$
  3. $\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ I(\boldsymbol{x}; \boldsymbol{a}) \leq 0 \wedge g(\boldsymbol{x}) > 0 \implies q(\boldsymbol{x}) \leq 0.$

  where PreCond$(\boldsymbol{x})$ is $p(\boldsymbol{x}) \leq 0$ and PostCond$(\boldsymbol{x})$ is $q(\boldsymbol{x}) \leq 0$.
- By applying **Putinar's Positivstellensatz**, for arbitrarily small $\epsilon > 0$

$$
\begin{aligned}
&\text{find} \quad \boldsymbol{a}, \boldsymbol{b} \\
-I(\boldsymbol{x}; \boldsymbol{a}) + \epsilon &= \sigma_1(\boldsymbol{x}; \boldsymbol{b}) + \sigma_2(\boldsymbol{x}; \boldsymbol{b}) \cdot (-p(\boldsymbol{x})), \\
-I(f(\boldsymbol{x}); \boldsymbol{a}) + \epsilon &= \sigma_3(\boldsymbol{x}; \boldsymbol{b}) + \sigma_4(\boldsymbol{x}; \boldsymbol{b}) \cdot (-g(\boldsymbol{x})) + \sigma_5(\boldsymbol{x}; \boldsymbol{b}) \cdot (-I(\boldsymbol{x}; \boldsymbol{a})) \\
-q(\boldsymbol{x}) + \epsilon &= \sigma_6(\boldsymbol{x}; \boldsymbol{b}) + \sigma_7(\boldsymbol{x}; \boldsymbol{b}) \cdot g(\boldsymbol{x}) + \sigma_8(\boldsymbol{x}; \boldsymbol{b}) \cdot (-I(\boldsymbol{x}; \boldsymbol{a})) \\
&\sigma_1(\boldsymbol{x}; \boldsymbol{b}), \ldots, \sigma_8(\boldsymbol{x}; \boldsymbol{b}) \in \Sigma[\boldsymbol{x}]
\end{aligned}
$$

SOS Constraints for Inductive Invariants

$$
\begin{aligned}
&\text{find} \quad \boldsymbol{a}, \boldsymbol{b} \\
-I(\boldsymbol{x}; \boldsymbol{a}) + \epsilon &= \sigma_1(\boldsymbol{x}; \boldsymbol{b}) + \sigma_2(\boldsymbol{x}; \boldsymbol{b}) \cdot (-p(\boldsymbol{x})), \\
-I(f(\boldsymbol{x}); \boldsymbol{a}) + \epsilon &= \sigma_3(\boldsymbol{x}; \boldsymbol{b}) + \sigma_4(\boldsymbol{x}; \boldsymbol{b}) \cdot (-g(\boldsymbol{x})) + \sigma_5(\boldsymbol{x}; \boldsymbol{b}) \cdot (-I(\boldsymbol{x}; \boldsymbol{a})) \\
-q(\boldsymbol{x}) + \epsilon &= \sigma_6(\boldsymbol{x}; \boldsymbol{b}) + \sigma_7(\boldsymbol{x}; \boldsymbol{b}) \cdot g(\boldsymbol{x}) + \sigma_8(\boldsymbol{x}; \boldsymbol{b}) \cdot (-I(\boldsymbol{x}; \boldsymbol{a})) \\
&\sigma_1(\boldsymbol{x}; \boldsymbol{b}), \ldots, \sigma_8(\boldsymbol{x}; \boldsymbol{b}) \in \Sigma[\boldsymbol{x}]
\end{aligned}
$$

- When σ_5, σ_8 is known, **linear matrix inequalities (LMIs)** $\Longrightarrow$ solved by **SDP solvers** efficiently.
- In general, **bilinear matrix inequalities (BMIs)** $\Longrightarrow$ non-convex and NP-hard.

SOS Constraints for Inductive Invariants

$$
\begin{aligned}
&\text{find} \quad \boldsymbol{a}, \boldsymbol{b} \\
-I(\boldsymbol{x}; \boldsymbol{a}) + \epsilon &= \sigma_1(\boldsymbol{x}; \boldsymbol{b}) + \sigma_2(\boldsymbol{x}; \boldsymbol{b}) \cdot (-p(\boldsymbol{x})), \\
-I(f(\boldsymbol{x}); \boldsymbol{a}) + \epsilon &= \sigma_3(\boldsymbol{x}; \boldsymbol{b}) + \sigma_4(\boldsymbol{x}; \boldsymbol{b}) \cdot (-g(\boldsymbol{x})) + \sigma_5(\boldsymbol{x}; \boldsymbol{b}) \cdot (-I(\boldsymbol{x}; \boldsymbol{a})) \\
-q(\boldsymbol{x}) + \epsilon &= \sigma_6(\boldsymbol{x}; \boldsymbol{b}) + \sigma_7(\boldsymbol{x}; \boldsymbol{b}) \cdot g(\boldsymbol{x}) + \sigma_8(\boldsymbol{x}; \boldsymbol{b}) \cdot (-I(\boldsymbol{x}; \boldsymbol{a})) \\
&\sigma_1(\boldsymbol{x}; \boldsymbol{b}), \ldots, \sigma_8(\boldsymbol{x}; \boldsymbol{b}) \in \Sigma[\boldsymbol{x}]
\end{aligned}
$$

- When σ_5, σ_8 is known, **linear matrix inequalities (LMIs)** $\Longrightarrow$ solved by **SDP solvers** efficiently.
- In general, **bilinear matrix inequalities (BMIs)** $\Longrightarrow$ non-convex and NP-hard.

SOS Constraints for Differential Invariants

- For differential invariants, similar:
  1. $\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ \boldsymbol{x} \in \mathcal{X}_i \implies B(\boldsymbol{x}; \boldsymbol{a}) \leq 0,$
  2. $\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ \boldsymbol{x} \in \mathcal{X}_d \wedge B(\boldsymbol{x}; \boldsymbol{a}) = 0 \implies \mathfrak{L}_{\boldsymbol{f}} B(\boldsymbol{x}; \boldsymbol{a}) \leq 0,$
  3. $\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ \boldsymbol{x} \in \mathcal{X}_u \implies B(\boldsymbol{x}; \boldsymbol{a}) > 0.$
- By applying **Putinar's Positivstellensatz**

$$\begin{aligned} \text{find} \quad & \boldsymbol{a}, \boldsymbol{b} \\ & -B(\boldsymbol{x}; \boldsymbol{a}) \in \mathbf{QM}(\mathcal{X}_i), \\ & \lambda(\boldsymbol{x}; \boldsymbol{b}) B(\boldsymbol{x}; \boldsymbol{a}) - \mathfrak{L}_{\boldsymbol{f}} B(\boldsymbol{x}; \boldsymbol{a}) \in \mathbf{QM}(\mathcal{X}_d) \\ & B(\boldsymbol{x}; \boldsymbol{a}) - \epsilon \in \mathbf{QM}(\mathcal{X}_u) \end{aligned}$$

- When $\lambda(\boldsymbol{x})$ is known, **linear matrix inequalities (LMIs)** $\implies$ solved by **SDP solvers** efficiently.
- In general, **bilinear matrix inequalities (BMIs)** $\implies$ non-convex and NP-hard.

SOS Constraints for Differential Invariants

- For differential invariants, similar:
  1. $\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ \boldsymbol{x} \in \mathcal{X}_i \implies B(\boldsymbol{x}; \boldsymbol{a}) \leq 0,$
  2. $\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ \boldsymbol{x} \in \mathcal{X}_d \wedge B(\boldsymbol{x}; \boldsymbol{a}) = 0 \implies \mathfrak{L}_{\boldsymbol{f}} B(\boldsymbol{x}; \boldsymbol{a}) \leq 0,$
  3. $\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ \boldsymbol{x} \in \mathcal{X}_u \implies B(\boldsymbol{x}; \boldsymbol{a}) > 0.$
- By applying **Putinar's Positivstellensatz**

$$
\begin{aligned}
\text{find} \quad & \boldsymbol{a}, \boldsymbol{b} \\
& -B(\boldsymbol{x}; \boldsymbol{a}) \in \mathbf{QM}(\mathcal{X}_i), \\
& \lambda(\boldsymbol{x}; \boldsymbol{b}) B(\boldsymbol{x}; \boldsymbol{a}) - \mathfrak{L}_{\boldsymbol{f}} B(\boldsymbol{x}; \boldsymbol{a}) \in \mathbf{QM}(\mathcal{X}_d) \\
& B(\boldsymbol{x}; \boldsymbol{a}) - \epsilon \in \mathbf{QM}(\mathcal{X}_u)
\end{aligned}
$$

- When $\lambda(x)$ is known, **linear matrix inequalities (LMIs)** $\implies$ solved by **SDP solvers** efficiently.
- In general, **bilinear matrix inequalities (BMIs)** $\implies$ non-convex and NP-hard.

SOS Constraints for Differential Invariants

- For differential invariants, similar:
  1. $\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ \boldsymbol{x} \in \mathcal{X}_i \implies B(\boldsymbol{x}; \boldsymbol{a}) \leq 0,$
  2. $\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ \boldsymbol{x} \in \mathcal{X}_d \wedge B(\boldsymbol{x}; \boldsymbol{a}) = 0 \implies \mathfrak{L}_{\boldsymbol{f}} B(\boldsymbol{x}; \boldsymbol{a}) \leq 0,$
  3. $\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ \boldsymbol{x} \in \mathcal{X}_u \implies B(\boldsymbol{x}; \boldsymbol{a}) > 0.$
- By applying **Putinar's Positivstellensatz**

$$\begin{aligned} \text{find} \quad & \boldsymbol{a}, \boldsymbol{b} \\ & -B(\boldsymbol{x}; \boldsymbol{a}) \in \mathbf{QM}(\mathcal{X}_i), \\ & \lambda(\boldsymbol{x}; \boldsymbol{b}) B(\boldsymbol{x}; \boldsymbol{a}) - \mathfrak{L}_{\boldsymbol{f}} B(\boldsymbol{x}; \boldsymbol{a}) \in \mathbf{QM}(\mathcal{X}_d) \\ & B(\boldsymbol{x}; \boldsymbol{a}) - \epsilon \in \mathbf{QM}(\mathcal{X}_u) \end{aligned}$$

- When $\lambda(\boldsymbol{x})$ is known, **linear matrix inequalities (LMIs)** $\implies$ solved by **SDP solvers** efficiently.
- In general, **bilinear matrix inequalities (BMIs)** $\implies$ non-convex and NP-hard.

Algebraic Tool: Homogenization

- Problem: Putinar's Positivstellensatz requires boundedness (i.e., the Archimedean condition). How to deal with **unbounded problems**?
- Idea: project the unbounded domain into a bounded region.
- Introduce a fresh variable x_0. Denote $\tilde{\boldsymbol{x}} = (x_0, \boldsymbol{x}) \in \mathbb{R}^{n+1}$.
- Let $p(\boldsymbol{x})$ be a polynomial of degree d, its **homogenization** is

$$\tilde{p}(\tilde{\boldsymbol{x}}) \triangleq x_0^d \cdot p(\frac{x_1}{x_0}, \ldots, \frac{x_n}{x_0}).$$

$$p(x_1, x_2) = x_1^2 + x_2 + 1 \implies \tilde{p}(x_0, x_1, x_2) = x_1^2 + x_2 x_0 + x_0^2.$$

- Let $S = \{\boldsymbol{x} \in \mathbb{R}^n \mid p_1(\boldsymbol{x}) \geq 0, \ldots, p_m(\boldsymbol{x}) \geq 0\}$, define

$$\tilde{S} \triangleq \{\tilde{\boldsymbol{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\boldsymbol{x}}) \geq 0, \ldots, \tilde{p}_m(\tilde{\boldsymbol{x}}) \geq 0, \underbrace{x_0 \geq 0, \|\tilde{\boldsymbol{x}}\|^2 = 1}_{\text{half unit sphere in } \mathbb{R}^{n+1}}\}.$$

Algebraic Tool: Homogenization

- Problem: Putinar's Positivstellensatz requires boundedness (i.e., the Archimedean condition). How to deal with **unbounded problems**?
- Idea: project the unbounded domain into a bounded region.
- Introduce a fresh variable x_0. Denote $\tilde{\boldsymbol{x}} = (x_0, \boldsymbol{x}) \in \mathbb{R}^{n+1}$.
- Let $p(\boldsymbol{x})$ be a polynomial of degree d, its **homogenization** is

$$\tilde{p}(\tilde{\boldsymbol{x}}) \triangleq x_0^d \cdot p(\frac{x_1}{x_0}, \ldots, \frac{x_n}{x_0}).$$

$$p(x_1, x_2) = x_1^2 + x_2 + 1 \implies \tilde{p}(x_0, x_1, x_2) = x_1^2 + x_2 x_0 + x_0^2.$$

- Let $S = \{\boldsymbol{x} \in \mathbb{R}^n \mid p_1(\boldsymbol{x}) \geq 0, \ldots, p_m(\boldsymbol{x}) \geq 0\}$, define

$$\tilde{S} \triangleq \{\tilde{\boldsymbol{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\boldsymbol{x}}) \geq 0, \ldots, \tilde{p}_m(\tilde{\boldsymbol{x}}) \geq 0, \underbrace{x_0 \geq 0, \|\tilde{\boldsymbol{x}}\|^2 = 1}_{\text{half unit sphere in } \mathbb{R}^{n+1}}\}.$$

Algebraic Tool: Homogenization

- Problem: Putinar's Positivstellensatz requires boundedness (i.e., the Archimedean condition). How to deal with **unbounded problems**?
- Idea: project the unbounded domain into a bounded region.
- Introduce a fresh variable x_0. Denote $\tilde{\boldsymbol{x}} = (x_0, \boldsymbol{x}) \in \mathbb{R}^{n+1}$.
- Let $p(\boldsymbol{x})$ be a polynomial of degree d, its **homogenization** is

$$\tilde{p}(\tilde{\boldsymbol{x}}) \triangleq x_0^d \cdot p(\frac{x_1}{x_0}, \ldots, \frac{x_n}{x_0}).$$

$$p(x_1, x_2) = x_1^2 + x_2 + 1 \implies \tilde{p}(x_0, x_1, x_2) = x_1^2 + x_2 x_0 + x_0^2.$$

- Let $S = \{\boldsymbol{x} \in \mathbb{R}^n \mid p_1(\boldsymbol{x}) \geq 0, \ldots, p_m(\boldsymbol{x}) \geq 0\}$, define

$$\tilde{S} \triangleq \{\tilde{\boldsymbol{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\boldsymbol{x}}) \geq 0, \ldots, \tilde{p}_m(\tilde{\boldsymbol{x}}) \geq 0, \underbrace{x_0 \geq 0, \|\tilde{\boldsymbol{x}}\|^2 = 1}_{\text{half unit sphere in } \mathbb{R}^{n+1}}\}.$$

Algebraic Tool: Homogenization

- Problem: Putinar's Positivstellensatz requires boundedness (i.e., the Archimedean condition). How to deal with **unbounded problems**?
- Idea: project the unbounded domain into a bounded region.
- Introduce a fresh variable x_0. Denote $\tilde{\boldsymbol{x}} = (x_0, \boldsymbol{x}) \in \mathbb{R}^{n+1}$.
- Let $p(\boldsymbol{x})$ be a polynomial of degree d, its **homogenization** is

$$\tilde{p}(\tilde{\boldsymbol{x}}) \triangleq x_0^d \cdot p(\frac{x_1}{x_0}, \ldots, \frac{x_n}{x_0}).$$

$$p(x_1, x_2) = x_1^2 + x_2 + 1 \implies \tilde{p}(x_0, x_1, x_2) = x_1^2 + x_2 x_0 + x_0^2.$$

- Let $S = \{\boldsymbol{x} \in \mathbb{R}^n \mid p_1(\boldsymbol{x}) \geq 0, \ldots, p_m(\boldsymbol{x}) \geq 0\}$, define

$$\tilde{S} \triangleq \{\tilde{\boldsymbol{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\boldsymbol{x}}) \geq 0, \ldots, \tilde{p}_m(\tilde{\boldsymbol{x}}) \geq 0, \underbrace{x_0 \geq 0, \|\tilde{\boldsymbol{x}}\|^2 = 1}_{\text{half unit sphere in } \mathbb{R}^{n+1}}\}.$$

Algebraic Tool: Homogenization

- Problem: Putinar's Positivstellensatz requires boundedness (i.e., the Archimedean condition). How to deal with **unbounded problems**?
- Idea: project the unbounded domain into a bounded region.
- Introduce a fresh variable x_0. Denote $\tilde{\boldsymbol{x}} = (x_0, \boldsymbol{x}) \in \mathbb{R}^{n+1}$.
- Let $p(\boldsymbol{x})$ be a polynomial of degree d, its **homogenization** is

$$\tilde{p}(\tilde{\boldsymbol{x}}) \;\hat{=}\; x_0^d \cdot p(\frac{x_1}{x_0}, \ldots, \frac{x_n}{x_0}).$$

$$p(x_1, x_2) = x_1^2 + x_2 + 1 \implies \tilde{p}(x_0, x_1, x_2) = x_1^2 + x_2 x_0 + x_0^2.$$

- Let $S = \{\boldsymbol{x} \in \mathbb{R}^n \mid p_1(\boldsymbol{x}) \geq 0, \ldots, p_m(\boldsymbol{x}) \geq 0\}$, define

$$\tilde{S} \;\hat{=}\; \{\tilde{\boldsymbol{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\boldsymbol{x}}) \geq 0, \ldots, \tilde{p}_m(\tilde{\boldsymbol{x}}) \geq 0, \underbrace{x_0 \geq 0, \|\tilde{\boldsymbol{x}}\|^2 = 1}_{\text{half unit sphere in } \mathbb{R}^{n+1}}\}.$$

Algebraic Tool: Homogenization

Theorem (Equivalence (Huang et al., 2023))

When a basic semialgebraic set S is closed at ∞,

$$f(\boldsymbol{x}) \geq 0 \text{ over } S \iff \tilde{f}(\tilde{\boldsymbol{x}}) \geq 0 \text{ over } \tilde{S}$$

- The quadratic module $\mathbf{QM}(\tilde{S})$ is Archimedean.

Complete SOS constraints for differential invariants.

- Old SOS constraints for exponential-type conditions:

$$
\begin{aligned}
&-B(\boldsymbol{x}) \in \mathbf{QM}(\mathcal{X}_i),\\
&B(\boldsymbol{x}) - \epsilon \in \mathbf{QM}(\mathcal{X}_u)\\
&\lambda(\boldsymbol{x})B(\boldsymbol{x}) - \mathfrak{L}_{\boldsymbol{f}}B(\boldsymbol{x}) \in \mathbf{QM}(\mathcal{X}_d)
\end{aligned}
$$

- First **homogenization**, then **Putinar's Positivstellensatz**.

$$
\begin{aligned}
&-\tilde{B}(x_0,\boldsymbol{x}) \in \mathbf{QM}(\tilde{\mathcal{X}}_i)\\
&\tilde{B}(x_0,\boldsymbol{x}) - \epsilon x_0^d \in \mathbf{QM}(\tilde{\mathcal{X}}_u)\\
&\tilde{H}(x_0,\boldsymbol{x}) \in \mathbf{QM}(\tilde{\mathcal{X}}_d)
\end{aligned}
$$

  where $H(\boldsymbol{x}) = \lambda(\boldsymbol{x})B(\boldsymbol{x}) - \mathfrak{L}_{\boldsymbol{f}}B(\boldsymbol{x})$.

- Assume that $\mathcal{X}_i$, $\mathcal{X}_u$, and $\mathcal{X}_d$ are all closed at infinity. Under some mild conditions, both sound and complete.

Complete SOS constraints for differential invariants.

- Old SOS constraints for exponential-type conditions:

$$
\begin{aligned}
&-B(\boldsymbol{x}) \in \mathbf{QM}(\mathcal{X}_i),\\
&B(\boldsymbol{x}) - \epsilon \in \mathbf{QM}(\mathcal{X}_u)\\
&\lambda(\boldsymbol{x})B(\boldsymbol{x}) - \mathfrak{L}_{\boldsymbol{f}}B(\boldsymbol{x}) \in \mathbf{QM}(\mathcal{X}_d)
\end{aligned}
$$

- First **homogenization**, then **Putinar's Positivstellensatz**.

$$
\begin{aligned}
&-\tilde{B}(x_0, \boldsymbol{x}) \in \mathbf{QM}(\tilde{\mathcal{X}}_i)\\
&\tilde{B}(x_0, \boldsymbol{x}) - \epsilon x_0^d \in \mathbf{QM}(\tilde{\mathcal{X}}_u)\\
&\tilde{H}(x_0, \boldsymbol{x}) \in \mathbf{QM}(\tilde{\mathcal{X}}_d)
\end{aligned}
$$

  where $H(\boldsymbol{x}) = \lambda(\boldsymbol{x})B(\boldsymbol{x}) - \mathfrak{L}_{\boldsymbol{f}}B(\boldsymbol{x})$.

- Assume that $\mathcal{X}_i$, $\mathcal{X}_u$, and $\mathcal{X}_d$ are all closed at infinity. Under some mild conditions, both sound and complete.

Complete SOS constraints for differential invariants.

- Old SOS constraints for exponential-type conditions:

$$
\begin{aligned}
&-B(\boldsymbol{x}) \in \mathbf{QM}(\mathcal{X}_i),\\
&B(\boldsymbol{x}) - \epsilon \in \mathbf{QM}(\mathcal{X}_u)\\
&\lambda(\boldsymbol{x})B(\boldsymbol{x}) - \mathfrak{L}_{\boldsymbol{f}}B(\boldsymbol{x}) \in \mathbf{QM}(\mathcal{X}_d)
\end{aligned}
$$

- First **homogenization**, then **Putinar's Positivstellensatz**.

$$
\begin{aligned}
&-\tilde{B}(x_0, \boldsymbol{x}) \in \mathbf{QM}(\tilde{\mathcal{X}}_i)\\
&\tilde{B}(x_0, \boldsymbol{x}) - \epsilon x_0^d \in \mathbf{QM}(\tilde{\mathcal{X}}_u)\\
&\tilde{H}(x_0, \boldsymbol{x}) \in \mathbf{QM}(\tilde{\mathcal{X}}_d)
\end{aligned}
$$

  where $H(\boldsymbol{x}) = \lambda(\boldsymbol{x})B(\boldsymbol{x}) - \mathfrak{L}_{\boldsymbol{f}}B(\boldsymbol{x})$.

- Assume that $\mathcal{X}_i$, $\mathcal{X}_u$, and $\mathcal{X}_d$ are all closed at infinity. Under some mild conditions, both sound and complete.

Weak Invariants vs Strong Invariants

- Fix an invariant template parameterized by $\boldsymbol{a}$.
- The **weak invariant synthesis problem** asks for one invariant satisfying the template, i.e., finding a valid parameter assignment.
- The **strong invariant synthesis problem** asks for a characterization of all possible invariants satisfying the template, i.e., characterizing the set of valid parameter assignments.

Problem Formulation

- We want to solve constraints of the form

$$\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ \bigwedge_{i=1}^{m} k_i(\boldsymbol{a}, \boldsymbol{x}) \geq 0 \implies l(\boldsymbol{a}, \boldsymbol{x}) \geq 0$$

  where we treat parameters $\boldsymbol{a}$ as variables and assume $\boldsymbol{x} \in C_{\boldsymbol{x}}$ and $\boldsymbol{a} \in C_{\boldsymbol{a}}$.

- Problem: To characterize the set of solutions for a, denoted by R.

Problem Formulation

- We want to solve constraints of the form

$$\exists \boldsymbol{a}, \forall \boldsymbol{x}.\ \bigwedge_{i=1}^{m} k_i(\boldsymbol{a}, \boldsymbol{x}) \geq 0 \implies l(\boldsymbol{a}, \boldsymbol{x}) \geq 0$$

  where we treat parameters $\boldsymbol{a}$ as variables and assume $\boldsymbol{x} \in C_{\boldsymbol{x}}$ and $\boldsymbol{a} \in C_{\boldsymbol{a}}$.

- Problem: To characterize the set of solutions for $\boldsymbol{a}$, denoted by R.

Representing Functions

- R can be expressed as

$$R = \{\boldsymbol{a} \mid \phi(\boldsymbol{a}) \leq 0\}.$$

with

$$\phi(a) = \begin{cases} -M & \textbf{if } K(a) = \emptyset, \\ \max(-M, \sup_{x \in K(a)} l(a, x)) & \text{otherwise,} \end{cases}$$

where $K(\boldsymbol{a}) = \{x \in C_x \mid \bigwedge_{i=1}^{m} k_i(\boldsymbol{a}, \boldsymbol{x}) \leq 0\}$ and $M > 0$ is a constant.

- The function ϕ is upper-semicontinuous, i.e., can be approximated from above by polynomials

Representing Functions

- R can be expressed as

$$R = \{\boldsymbol{a} \mid \phi(\boldsymbol{a}) \leq 0\}.$$

with

$$\phi(a) = \begin{cases} -M & \textbf{if } K(a) = \emptyset, \\ \max(-M, \sup_{x \in K(a)} l(a, x)) & \text{otherwise,} \end{cases}$$

where $K(\boldsymbol{a}) = \{x \in C_x \mid \bigwedge_{i=1}^{m} k_i(\boldsymbol{a}, \boldsymbol{x}) \leq 0\}$ and $M > 0$ is a constant.

- The function ϕ is upper-semicontinuous, i.e., can be approximated from above by polynomials

Under-approximating R

- Under-approximations of set $R = \{\boldsymbol{a} \mid \phi(\boldsymbol{a}) \leq 0\}$ can be obtained by approximating ϕ from above:

$$p_i(\boldsymbol{a}) \geq \phi(\boldsymbol{a}) \implies R_i = \{\boldsymbol{a} \mid p_i(\boldsymbol{a}) \leq 0\} \subseteq R$$

Intuitively, the better p_i approximates ϕ, the less conservative R_i approximates R.

Theorem (Lasserre, 2015)

Assume that $\partial R = \{a \mid \phi(a) = 0\}$ has Lebesgue measure zero. If $p_i \geq \phi$ converges to ϕ w.r.t. L1-norm, then $R_i \subseteq R$ and:

$$\lim_{i \to \infty} \mu(R \setminus R_i) = 0 \qquad (\mu \text{ Lebesgue measure})$$

Under-approximation by SDP

- Optimization formulation:

$$\begin{aligned} \min \quad & \sum_{\alpha} \gamma_\alpha \mathrm{p}_\alpha^i \\ s.t. \quad & p_i(\boldsymbol{a}) - l(\boldsymbol{a}, \boldsymbol{x}) \geq 0,\ \forall (\boldsymbol{a}, \boldsymbol{x}) \in C_a \times K_i(\boldsymbol{a}) \\ & p_i(\boldsymbol{a}) + M \geq 0,\ \forall (\boldsymbol{a}, \boldsymbol{x}) \in C_a \times C_{\boldsymbol{x}}. \end{aligned}$$

  where the objective function means searching for p_i with minimal L1-norm over $C_{\boldsymbol{a}}$.

- By applying **Putinar's Positivstellensatz**, the problem can be directly solved by SDP.

Under-approximation by SDP

- Optimization formulation:

$$\begin{aligned} \min \quad & \sum_{\alpha} \gamma_\alpha \mathrm{p}_\alpha^i \\ s.t. \quad & p_i(\boldsymbol{a}) - l(\boldsymbol{a}, \boldsymbol{x}) \geq 0,\ \forall (\boldsymbol{a}, \boldsymbol{x}) \in C_a \times K_i(\boldsymbol{a}) \\ & p_i(\boldsymbol{a}) + M \geq 0,\ \forall (\boldsymbol{a}, \boldsymbol{x}) \in C_a \times C_{\boldsymbol{x}}. \end{aligned}$$

  where the objective function means searching for p_i with minimal L1-norm over $C_{\boldsymbol{a}}$.

- By applying **Putinar's Positivstellensatz**, the problem can be directly solved by SDP.

Problem Formulation

- Problem: How to solve Bilinear Matrix Inequalities (BMIs) faster?
- Idea: Utilize the Difference-of-Convex (DC) framework in non-convex optimization.
- Target BMI constraints:

$$\max_{\boldsymbol{x},\boldsymbol{y}} \quad \mu(\boldsymbol{x},\boldsymbol{y})$$

$$s.t. \quad \mathcal{B}(\boldsymbol{x},\boldsymbol{y}) = \begin{pmatrix} \boldsymbol{x}\otimes I \\ \boldsymbol{y}\otimes I \end{pmatrix}^{\mathsf{T}} M \begin{pmatrix} \boldsymbol{x}\otimes I \\ \boldsymbol{y}\otimes I \end{pmatrix} + \begin{pmatrix} \Omega_1 & \Omega_2 \end{pmatrix} \begin{pmatrix} \boldsymbol{x}\otimes I \\ \boldsymbol{y}\otimes I \end{pmatrix} + F \preceq 0$$

where $\boldsymbol{x} \in \mathbb{R}^m$, $\boldsymbol{y} \in \mathbb{R}^n$, μ is linear, and M, Ω_1, Ω_2, F are matrices.

- Note that M is symmetric but not necessarily PSD.

Problem Formulation

- Problem: How to solve Bilinear Matrix Inequalities (BMIs) faster?
- Idea: Utilize the Difference-of-Convex (DC) framework in non-convex optimization.
- Target BMI constraints:

$$\max_{\boldsymbol{x},\boldsymbol{y}} \quad \mu(\boldsymbol{x},\boldsymbol{y})$$

$$s.t. \quad \mathcal{B}(\boldsymbol{x},\boldsymbol{y}) = \begin{pmatrix} \boldsymbol{x}\otimes I \\ \boldsymbol{y}\otimes I \end{pmatrix}^{\mathsf{T}} M \begin{pmatrix} \boldsymbol{x}\otimes I \\ \boldsymbol{y}\otimes I \end{pmatrix} + \begin{pmatrix} \Omega_1 & \Omega_2 \end{pmatrix} \begin{pmatrix} \boldsymbol{x}\otimes I \\ \boldsymbol{y}\otimes I \end{pmatrix} + F \preceq 0$$

where $\boldsymbol{x} \in \mathbb{R}^m$, $\boldsymbol{y} \in \mathbb{R}^n$, μ is linear, and M, Ω_1, Ω_2, F are matrices.

- Note that M is symmetric but not necessarily PSD.

Problem Formulation

- Problem: How to solve Bilinear Matrix Inequalities (BMIs) faster?
- Idea: Utilize the Difference-of-Convex (DC) framework in non-convex optimization.
- Target BMI constraints:

$$\begin{aligned} \max_{\boldsymbol{x},\boldsymbol{y}} \quad & \mu(\boldsymbol{x},\boldsymbol{y}) \\ s.t. \quad & \mathcal{B}(\boldsymbol{x},\boldsymbol{y}) = \begin{pmatrix} \boldsymbol{x}\otimes I \\ \boldsymbol{y}\otimes I \end{pmatrix}^{\mathsf{T}} M \begin{pmatrix} \boldsymbol{x}\otimes I \\ \boldsymbol{y}\otimes I \end{pmatrix} + \begin{pmatrix} \Omega_1 & \Omega_2 \end{pmatrix} \begin{pmatrix} \boldsymbol{x}\otimes I \\ \boldsymbol{y}\otimes I \end{pmatrix} + F \preceq 0 \end{aligned}$$

where $\boldsymbol{x} \in \mathbb{R}^m$, $\boldsymbol{y} \in \mathbb{R}^n$, μ is linear, and M, Ω_1, Ω_2, F are matrices.

- Note that M is symmetric but not necessarily PSD.

Problem Formulation

- Problem: How to solve Bilinear Matrix Inequalities (BMIs) faster?
- Idea: Utilize the Difference-of-Convex (DC) framework in non-convex optimization.
- Target BMI constraints:

$$\begin{aligned} \max_{\boldsymbol{x},\boldsymbol{y}} \quad & \mu(\boldsymbol{x},\boldsymbol{y}) \\ s.t. \quad & \mathcal{B}(\boldsymbol{x},\boldsymbol{y}) = \begin{pmatrix} \boldsymbol{x}\otimes I \\ \boldsymbol{y}\otimes I \end{pmatrix}^{\mathsf{T}} M \begin{pmatrix} \boldsymbol{x}\otimes I \\ \boldsymbol{y}\otimes I \end{pmatrix} + \begin{pmatrix} \Omega_1 & \Omega_2 \end{pmatrix} \begin{pmatrix} \boldsymbol{x}\otimes I \\ \boldsymbol{y}\otimes I \end{pmatrix} + F \preceq 0 \end{aligned}$$

  where $\boldsymbol{x} \in \mathbb{R}^m$, $\boldsymbol{y} \in \mathbb{R}^n$, μ is linear, and M, Ω_1, Ω_2, F are matrices.
- Note that M is symmetric but not necessarily PSD.

Step 1: Difference-of-Convex Decomposition

- Decompose M as $M = M_1 - M_2$, where both M_1 and M_2 are PSD. For example, using **eigenvalue decomposition**.
- Then we have

$$\mathcal{B}(\boldsymbol{x}, \boldsymbol{y}) = \mathcal{B}^+(\boldsymbol{x}, \boldsymbol{y}) - \mathcal{B}^-(\boldsymbol{x}, \boldsymbol{y}),$$

where

$$\mathcal{B}^+(\boldsymbol{x}, \boldsymbol{y}) \triangleq \begin{pmatrix} \boldsymbol{x} \otimes I \\ \boldsymbol{y} \otimes I \end{pmatrix}^\intercal M_1 \begin{pmatrix} \boldsymbol{x} \otimes I \\ \boldsymbol{y} \otimes I \end{pmatrix} + \begin{pmatrix} \Omega_1 & \Omega_2 \end{pmatrix} \begin{pmatrix} \boldsymbol{x} \otimes I \\ \boldsymbol{y} \otimes I \end{pmatrix} + F,$$

$$\mathcal{B}^-(\boldsymbol{x}, \boldsymbol{y}) \triangleq \begin{pmatrix} \boldsymbol{x} \otimes I \\ \boldsymbol{y} \otimes I \end{pmatrix}^\intercal M_2 \begin{pmatrix} \boldsymbol{x} \otimes I \\ \boldsymbol{y} \otimes I \end{pmatrix}$$

are both PSD-convex.

Step 2: Successive Convexification

- At the $(k+1)$-th iteration, given the current feasible point $\boldsymbol{z}_k = (\boldsymbol{x}_k, \boldsymbol{y}_k)$, the next point $\boldsymbol{z}_{k+1}$ is obtained by solving the following SDP:

$$\begin{aligned} \max_{\boldsymbol{z}=(\boldsymbol{x},\boldsymbol{y})} \quad & \mu(\boldsymbol{z}) + \frac{1}{2}\delta\|\boldsymbol{z}-\boldsymbol{z}_k\|^2 \\ s.t. \quad & \mathcal{B}^+(\boldsymbol{z}) - \underbrace{\left(\mathcal{B}^-(\boldsymbol{z}_k) + \langle \nabla\mathcal{B}^-(\boldsymbol{z}_k), \boldsymbol{z}-\boldsymbol{z}_k\rangle\right)}_{\text{linearize } \mathcal{B}^- \text{ at } \boldsymbol{z}_k} \preceq 0, \end{aligned}$$

- Guarantee to converge to local optimum of the original BMI problem.
- To compute global optimum, need other techniques such as **branch and bound**.

Summary: Pros and Cons

- Key steps of optimization-based methods:
  1. Set a parameterized template, usually linear or polynomial.
  2. Encode the constraints for parameters using **Putinar's Theorem** or other representation theorems.
  3. Solve the constraints, in general non-convex.
- Pros:
  1. a unified framework for discrete-time and continuous-time.
  2. based on numerical optimization techniques, efficient.
  3. theoretical guarantee: relative completeness.
- Cons:
  1. How to find a good template?
  2. Still not scalable enough.
  3. Possible numerical errors, 2 vs 2.0001.
- Future Work: to fill the gap between theory and application.

Summary: Pros and Cons

- Key steps of optimization-based methods:
  1. Set a parameterized template, usually linear or polynomial.
  2. Encode the constraints for parameters using **Putinar's Theorem** or other representation theorems.
  3. Solve the constraints, in general non-convex.
- Pros:
  1. a unified framework for discrete-time and continuous-time.
  2. based on numerical optimization techniques, efficient.
  3. theoretical guarantee: relative completeness.
- Cons:
  1. How to find a good template?
  2. Still not scalable enough.
  3. Possible numerical errors, 2 vs 2.0001.
- Future Work: to fill the gap between theory and application.

Summary: Pros and Cons

- Key steps of optimization-based methods:
  1. Set a parameterized template, usually linear or polynomial.
  2. Encode the constraints for parameters using **Putinar's Theorem** or other representation theorems.
  3. Solve the constraints, in general non-convex.
- Pros:
  1. a unified framework for discrete-time and continuous-time.
  2. based on numerical optimization techniques, efficient.
  3. theoretical guarantee: relative completeness.
- Cons:
  1. How to find a good template?
  2. Still not scalable enough.
  3. Possible numerical errors, 2 vs 2.0001.
- Future Work: to fill the gap between theory and application.

Summary: Pros and Cons

- Key steps of optimization-based methods:
  1. Set a parameterized template, usually linear or polynomial.
  2. Encode the constraints for parameters using **Putinar's Theorem** or other representation theorems.
  3. Solve the constraints, in general non-convex.
- Pros:
  1. a unified framework for discrete-time and continuous-time.
  2. based on numerical optimization techniques, efficient.
  3. theoretical guarantee: relative completeness.
- Cons:
  1. How to find a good template?
  2. Still not scalable enough.
  3. Possible numerical errors, 2 vs 2.0001.
- Future Work: to fill the gap between theory and application.

Related Works

Thanks! & Questions?

- Qiuye Wang, Mingshuai Chen, Bai Xue, Naijun Zhan, Joost-Pieter Katoen: Synthesizing Invariant Barrier Certificates via Difference-of-Convex Programming. CAV'21
- Qiuye Wang, Mingshuai Chen, Bai Xue, Naijun Zhan, Joost-Pieter Katoen: Encoding inductive invariants as barrier certificates: Synthesis via difference-of-convex programming. I&C'22
- Hao Wu, Shenghua Feng, Naijun Zhan, Jie Wang, Bican Xia and Ting Gan: On Completeness of SDP-Based Barrier Certificate Synthesis over Unbounded Domains. FM'24
- Hao Wu, Qiuye Wang, Bai Xue, Naijun Zhan, Lihong Zhi, Zhihong Yang: Synthesizing Invariants for Polynomial Programs by Semidefinite Programming. TOPLAS'25